

COMP6237 Data Mining

Lecture 7: Decision Trees

Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

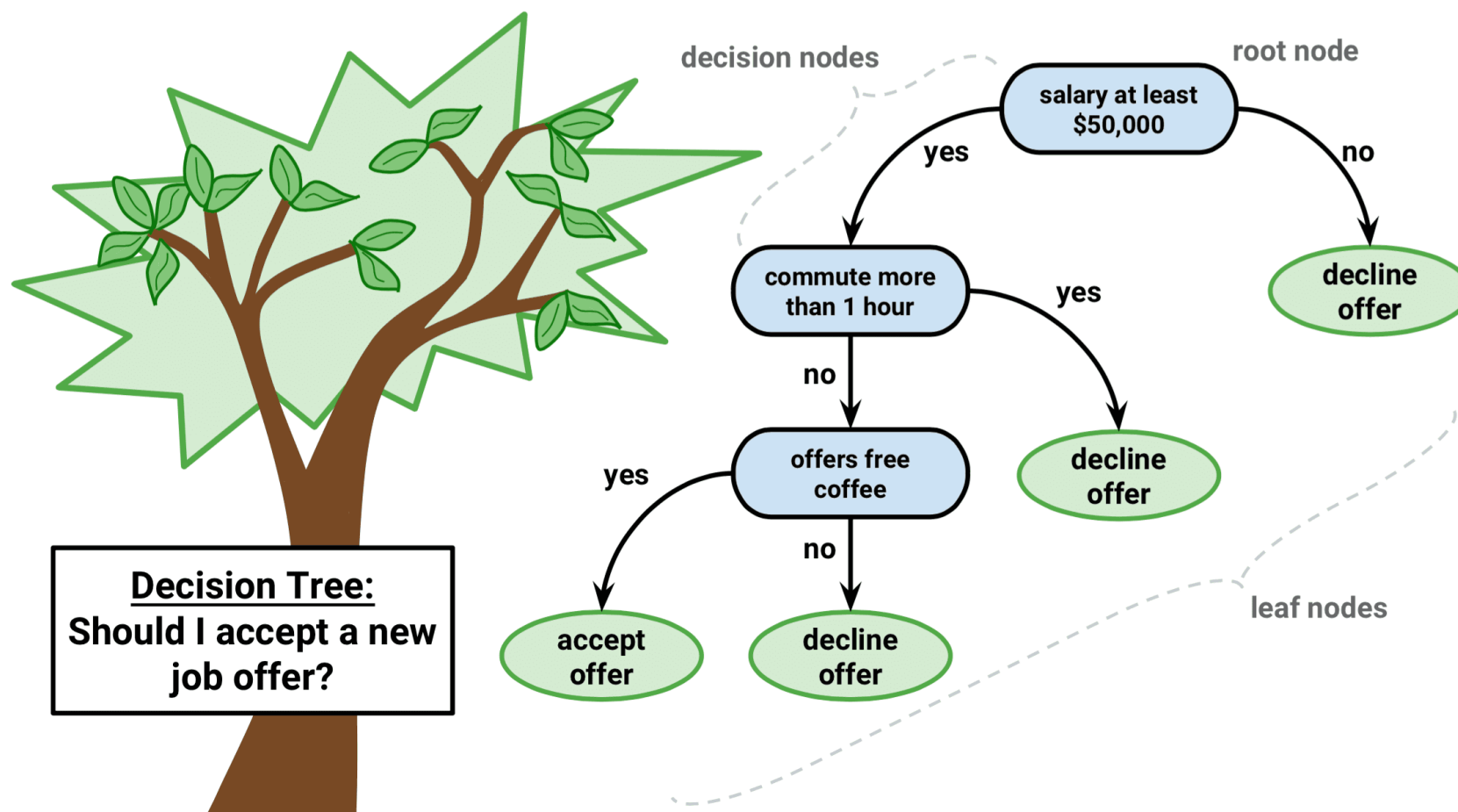
Lecturer (Assistant Professor) @ VLC of ECS
University of Southampton

Lecture slides available here:

<http://comp6237.ecs.soton.ac.uk/zh.html>

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

Decision Trees – Problem



Source: <http://dataaspirant.com/2017/01/30/how-decision-tree-algorithm-works/>

Decision Trees – Problem

Decision Trees can be 'hand crafted' by experts

They can also be built up using machine learning techniques

They are **interpretable**, it is easy to see how they made a certain decision

They are used in a wide range of contexts, for example:

- ▶ Medicine
- ▶ Financial Analysis
- ▶ Astronomy

Especially in medicine, the explicit reasoning in decision trees means experts can understand why the algorithm has made its decision.

Decision Trees – Approach: CART

The CART algorithm was published by Breiman *et al* in 1984

- ▶ Find best split for each feature - minimises impurity measure
- ▶ Find feature that minimises impurity the most
- ▶ Use the best split on that feature to split the node
- ▶ Do the same for each of the leaf nodes

The CART algorithm depends on an impurity measure. It uses Gini impurity

Gini impurity measures how often a randomly chosen element from a set would be incorrectly labelled if it was randomly labelled according to the distribution of labels in the set. The probabilities for each label are summed up.

Decision Trees – Approach: CART

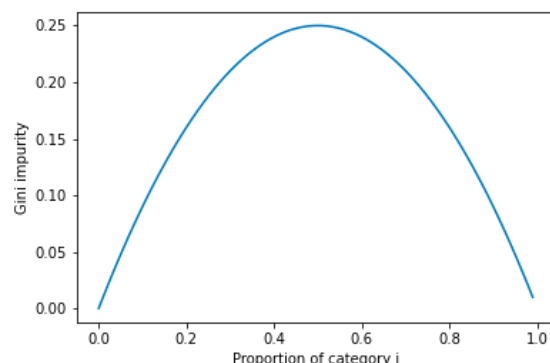
- Gini Index for a given node t :

The smaller
the purer

$$GINI(t) = 1 - \sum_j^{n_c} [p(j|t)]^2$$

(where: n_c is the number of classes, and $p(j | t)$ is the relative frequency of class j at node t).

- Maximum: $(1 - 1/n_c)$ when records are equally distributed among all classes, implying least interesting information
- Minimum: (0.0) when all records belong to one single class, implying most interesting information



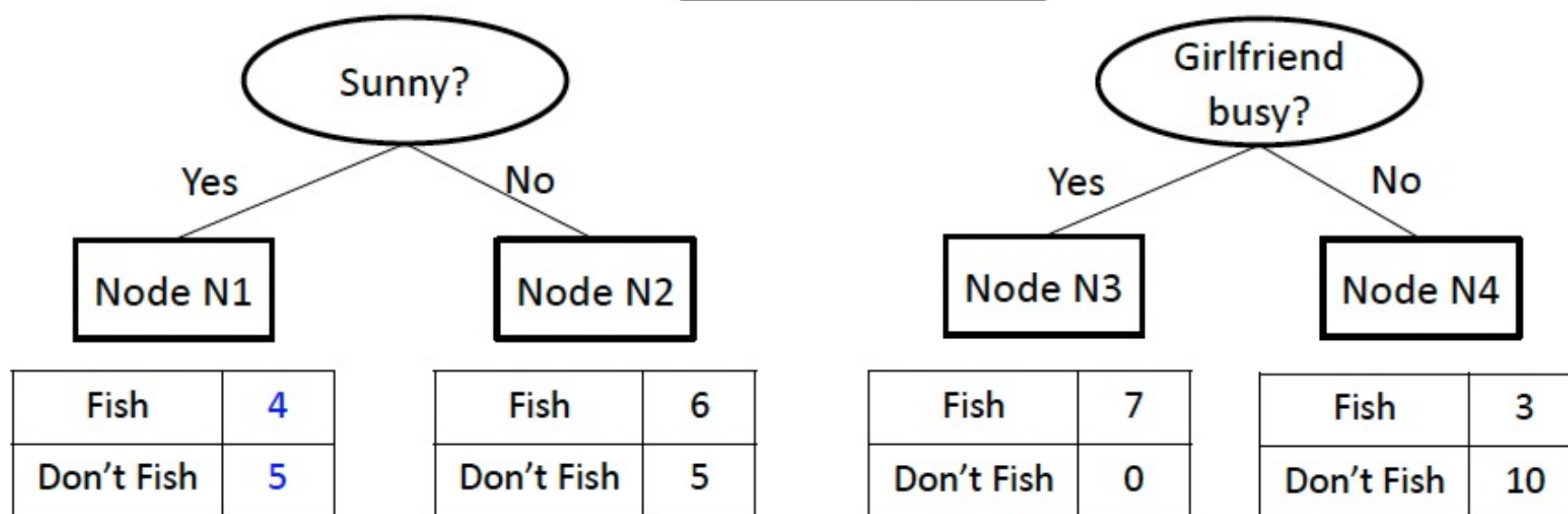
It reaches its minimum when all cases in the node fall into a single category

Decision Trees – Approach: CART

Before splitting

Fish	10
Don't Fish	10

*Note that “fish” or “don’t fish” is the class label



Using Gini Index, evaluate which test condition is better

Decision Trees – Approach: CART

$$GINI(t) = 1 - \sum_j^{n_c} [p(j|t)]^2$$

Before splitting

Fish	10
Don't Fish	10

$$\begin{aligned} \text{Gini} &= 1 - [p(\text{Fish})^2 + p(\text{Don't Fish})^2] \\ &= 1 - [(10/20)^2 + (10/20)^2] \\ &= 1 - (0.25 + 0.25) = 0.5 \end{aligned}$$

Split by "Sunny?"

Yes

Fish	4
Don't Fish	5

$$P(\text{Fish}) = 4/9 \quad P(\text{Don't Fish}) = 5/9$$

$$\text{Gini} = 1 - [(4/9)^2 + (5/9)^2] = 0.494 \text{ updated value}$$

No

Fish	6
Don't Fish	5

$$P(\text{Fish}) = 6/11 \quad P(\text{Don't Fish}) = 5/11$$

$$\text{Gini} = 1 - [(6/11)^2 + (5/11)^2] = 0.496$$

Question: How to aggregate the branch-based values for the whole split?

Decision Trees – CART

- When a node p is split into k partitions (children), the quality of split is computed as,

$$GINI_{split} = \sum_{i=1}^k \frac{n_i}{n} GINI(i) \quad (1)$$

where k is number of branches
 $GINI(i)$ is the i -th branch-based GINI
 n_i = number of records at child i ,
 n = number of records at node p .

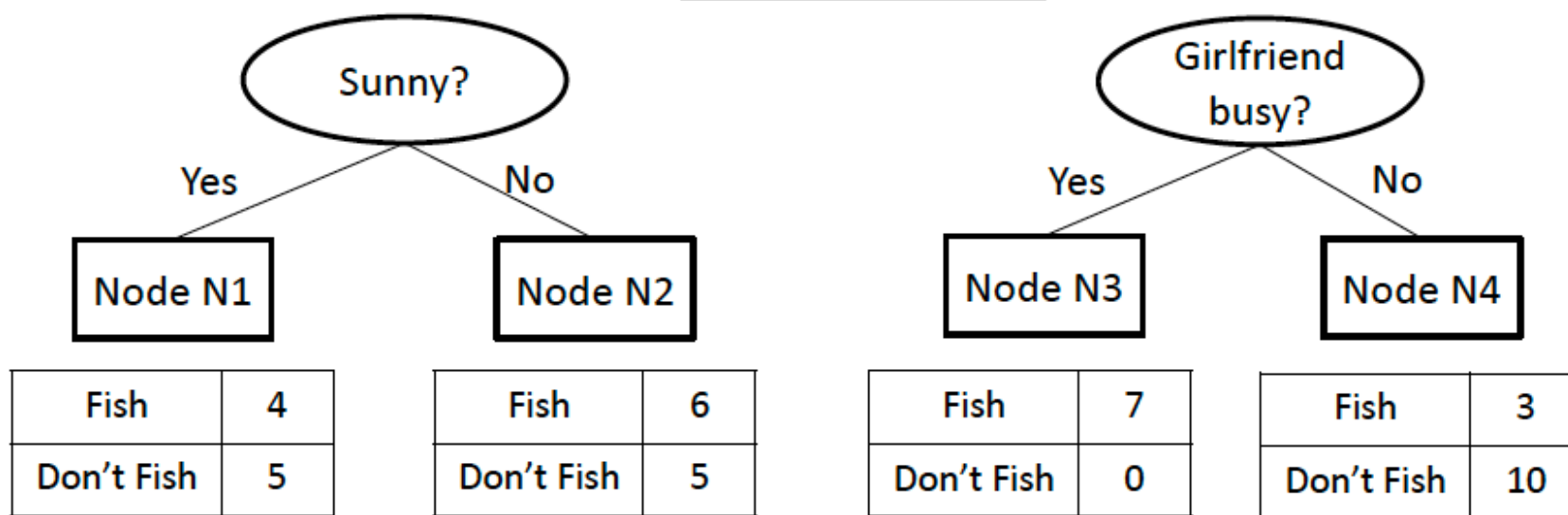
Information gain: $Gain_{split} = GINI_{original} - GINI_{split}$,
The higher gain, the better split

Decision Trees – Approach: CART

Before splitting

Fish	10
Don't Fish	10

*Note that “fish” or “don’t fish” is the class label



Gini(N1) = 0.494
(updated value)

Gini(N2) = 0.496

Gini(N3) = 0.0

Gini(N4) = 0.355

$$\begin{aligned} \text{Gini}_{\text{sunny}} &= (9/20 * 0.494) + \\ &\quad (11/20 * 0.496) \\ &= 0.495 \end{aligned}$$

$$\begin{aligned} \text{Gini}_{\text{girlfriend}} &= (7/20 * 0.0) + \\ &\quad (13/20 * 0.355) \\ &= 0.23 \end{aligned}$$

$$GINI_{\text{split}} = \sum_{i=1}^k \frac{n_i}{n} GINI(i) \quad (1)$$



$$\text{Gain}_{\text{split}} = GINI_{\text{original}} - GINI_{\text{split}} = 0.5 - GINI_{\text{split}}$$

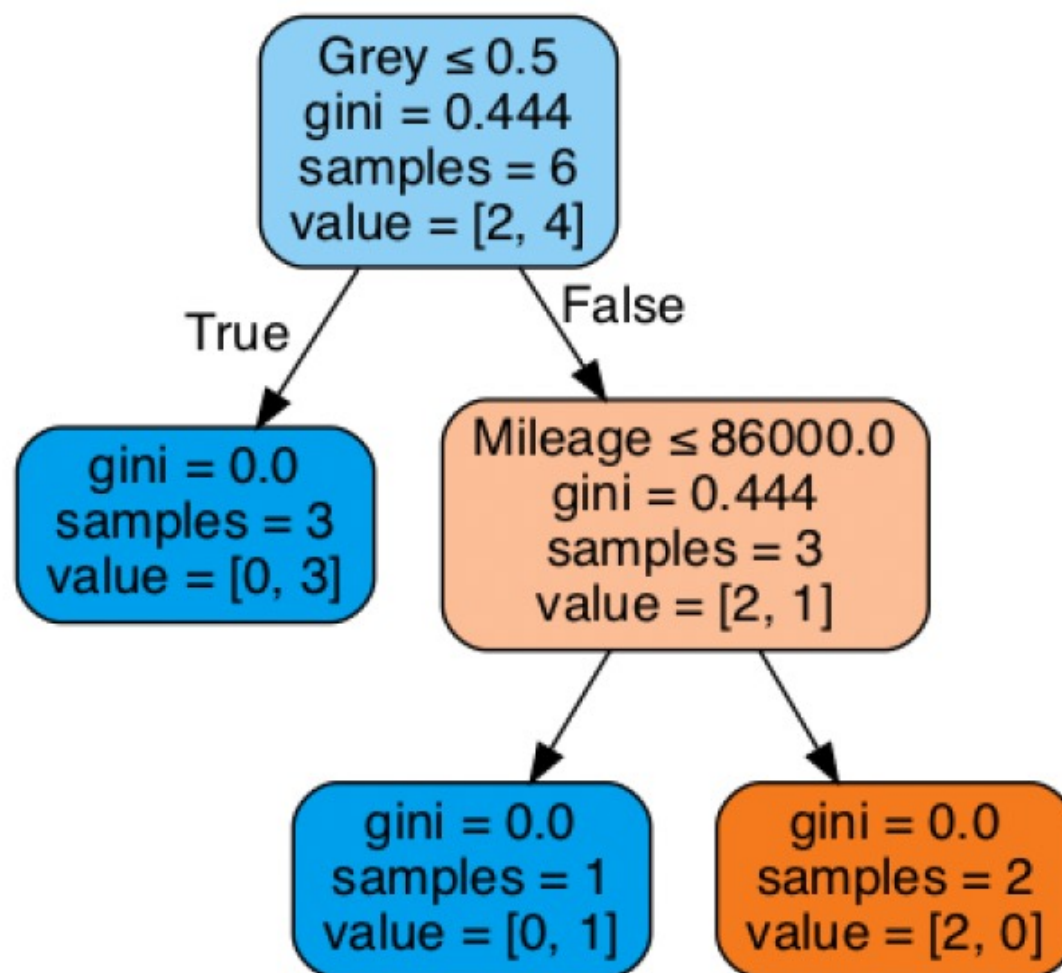
Decision Trees – CART

Example:

Car Make	Type	Colour	Price	Mileage	Bought?
VW	Polo	Grey	£2000	82000	Yes
Ford	Fiesta	Purple	£1795	95000	Yes
Ford	Fiesta	Grey	£1990	90000	No
VW	Golf	Red	£1800	120000	Yes
VW	Polo	Grey	£900	150000	No
Ford	Ka	Yellow	£1400	100000	Yes

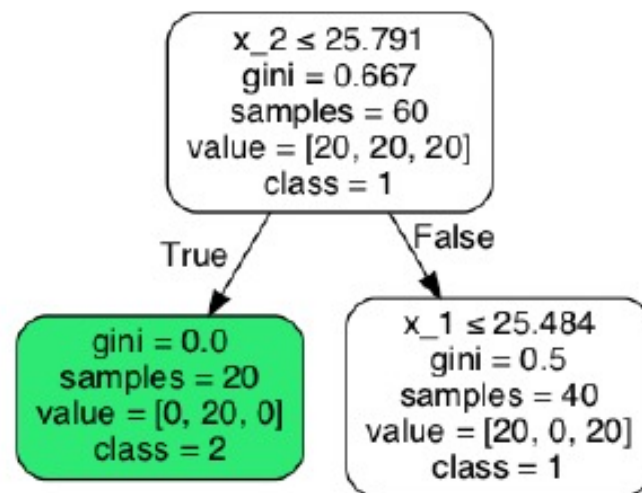
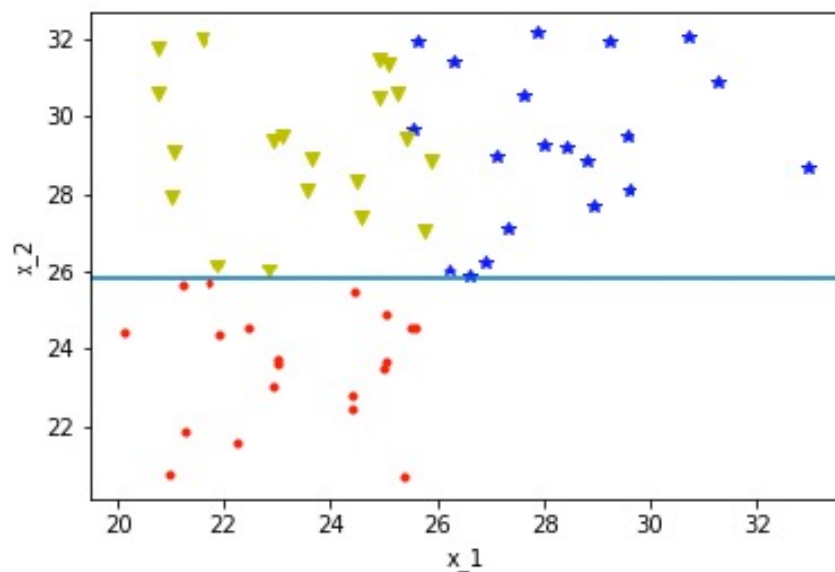
Can go through and calculate best split for each feature.

Decision Trees – CART

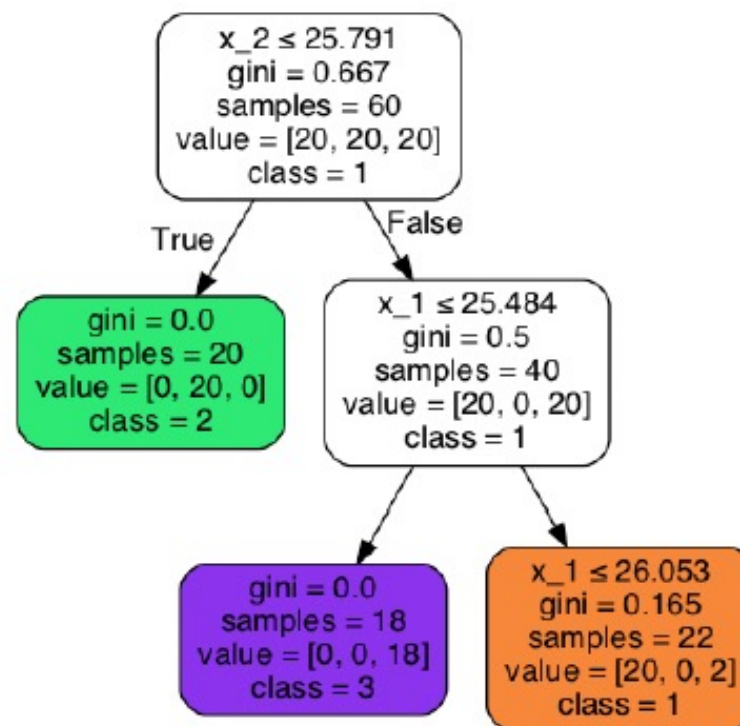
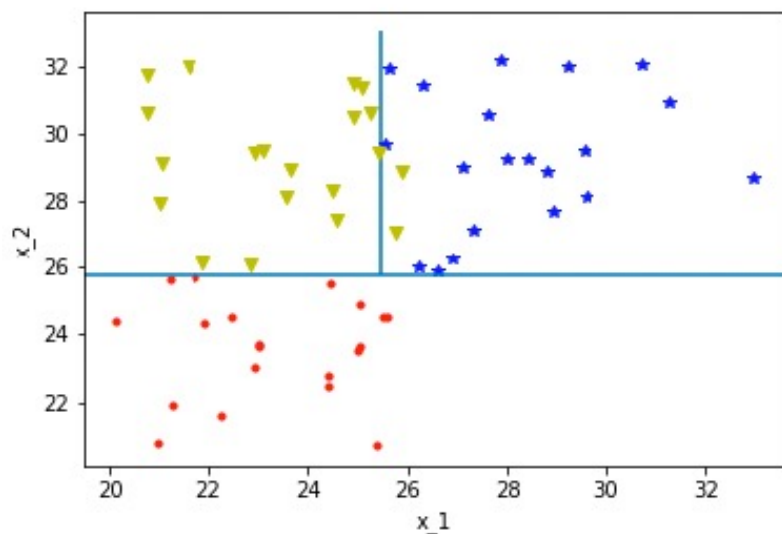


Decision Trees – CART

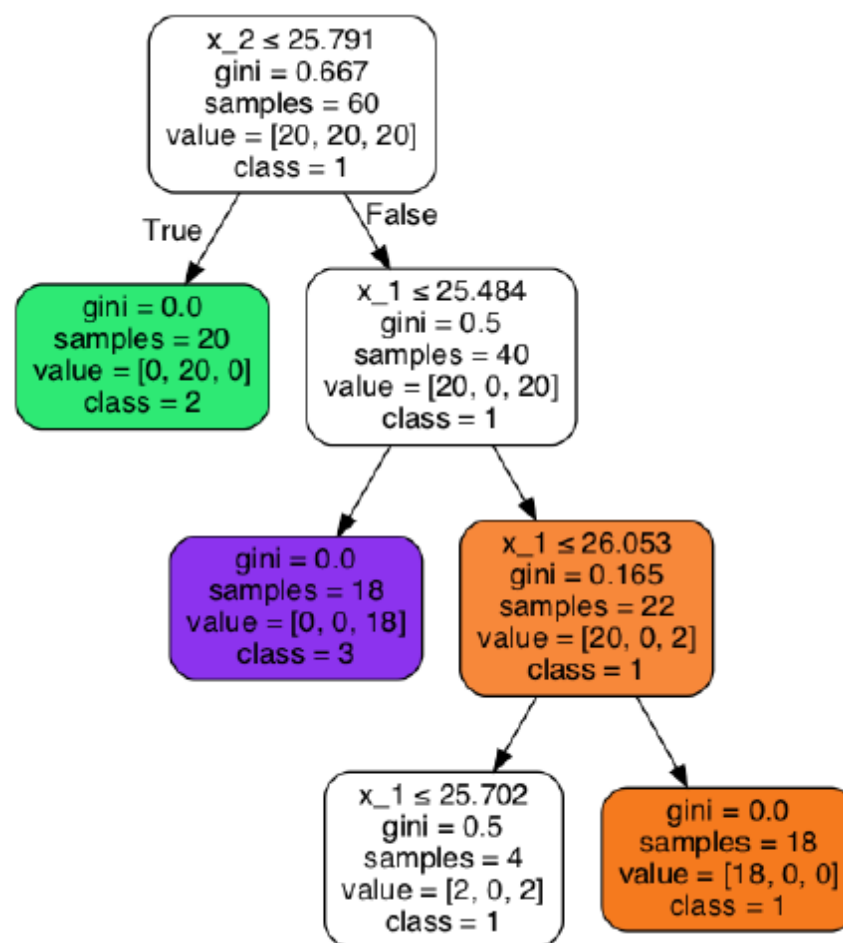
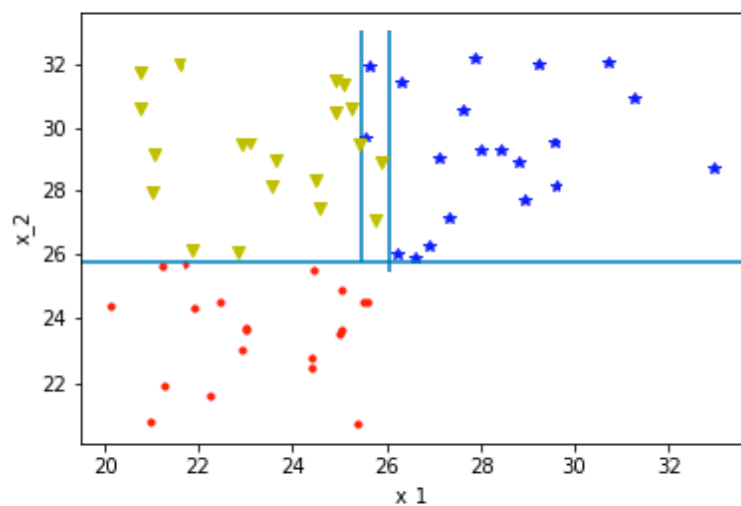
With three classes:



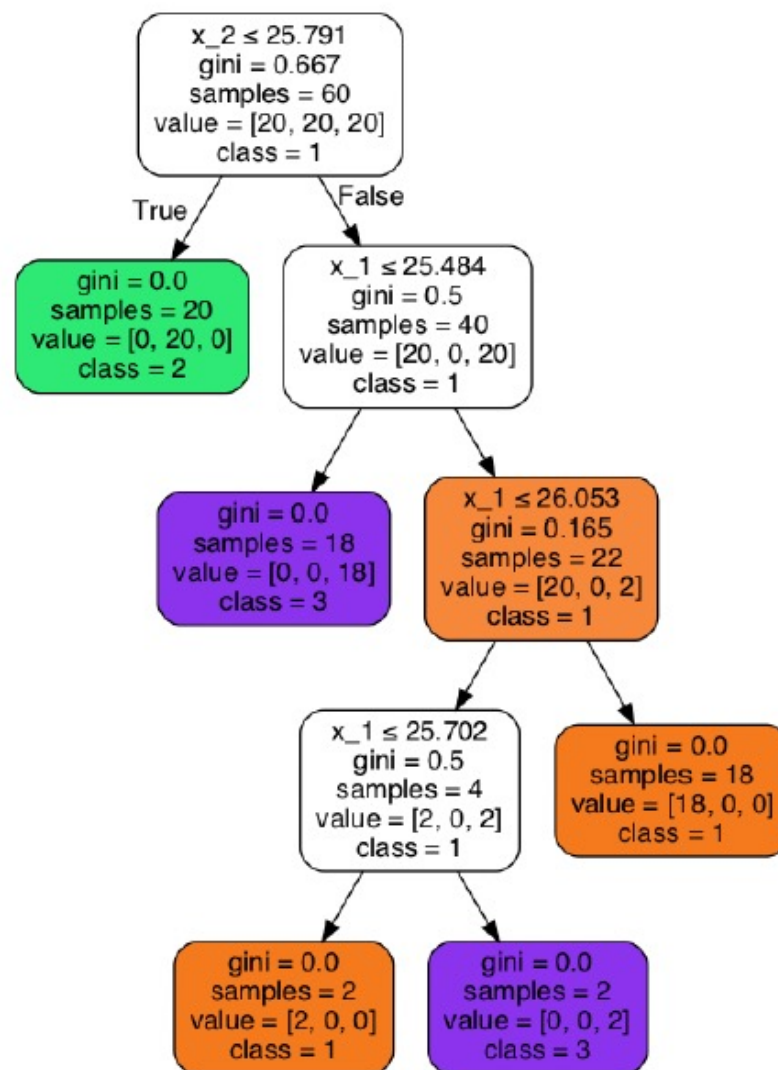
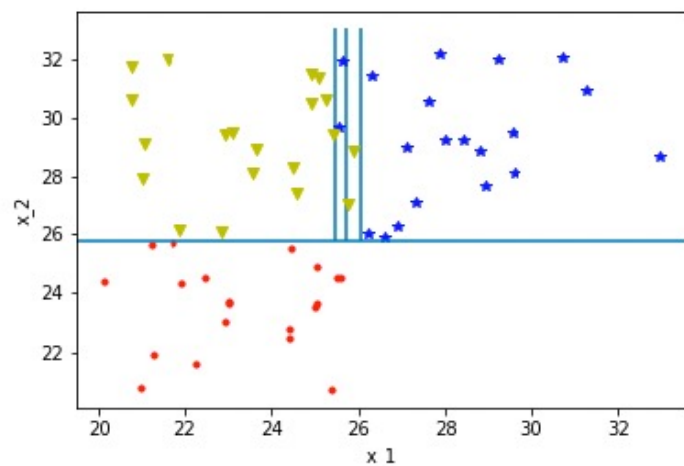
Decision Trees – CART



Decision Trees – CART



Decision Trees – CART



Overfitting..

Decision Trees – Approach: ID3

Similar to CART, Iterative Dichotomy 3 (ID3) minimises entropy

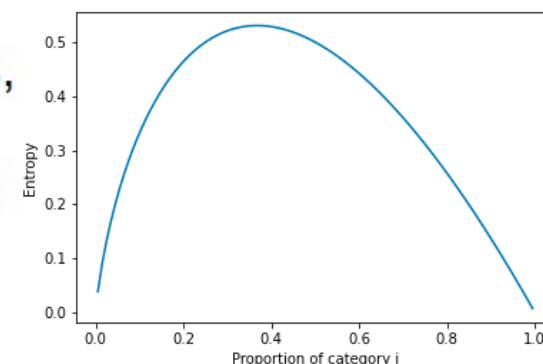
- Entropy at a given node t :

The smaller
the purer

$$Entropy(t) = - \sum_{j=1}^{n_c} p(j|t) \log_2 p(j|t) \quad (1)$$

(where: n_c is the number of classes, and $p(j | t)$ is the relative frequency of class j at node t).

- Measures **homogeneity** of a node.
 - Maximum ($\log_2 n_c$) when records are equally distributed among all classes implying the **least** information
 - Minimum (**0.0**) when all records belong to one class, implying the **most** information
- Entropy based computations are similar to the GINI index computations



Decision Trees – Approach: ID3

The higher
the better

- Information Gain:

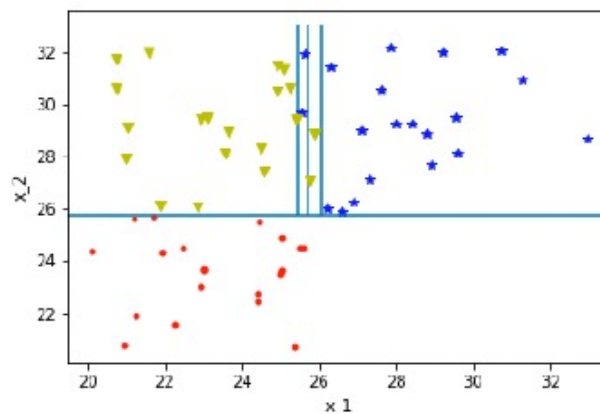
$$GAIN_{split} = Entropy(p) - \left(\sum_{i=1}^k \frac{n_i}{n} Entropy(i) \right) \quad (1)$$

where $Entropy(p)$: entropy of Parent Node, $Entropy(i)$:
entropy of the i -th branch/partition, n is split into k partitions;
 n_i is number of records in partition i

- Measures **Reduction in Entropy** achieved because of the split. Choose the split that achieves most reduction (maximizes GAIN)
- Used in ID3 and C4.5

Decision Trees – Address Overfitting Pruning

Overfitting is a serious problem with Decision Trees



.. are four splits really required here?

The trees it creates are too complex.

One solution is **pruning**

This can be done in a variety of ways, including:

- ▶ Reduced Error Pruning
- ▶ Entropy Based Merging

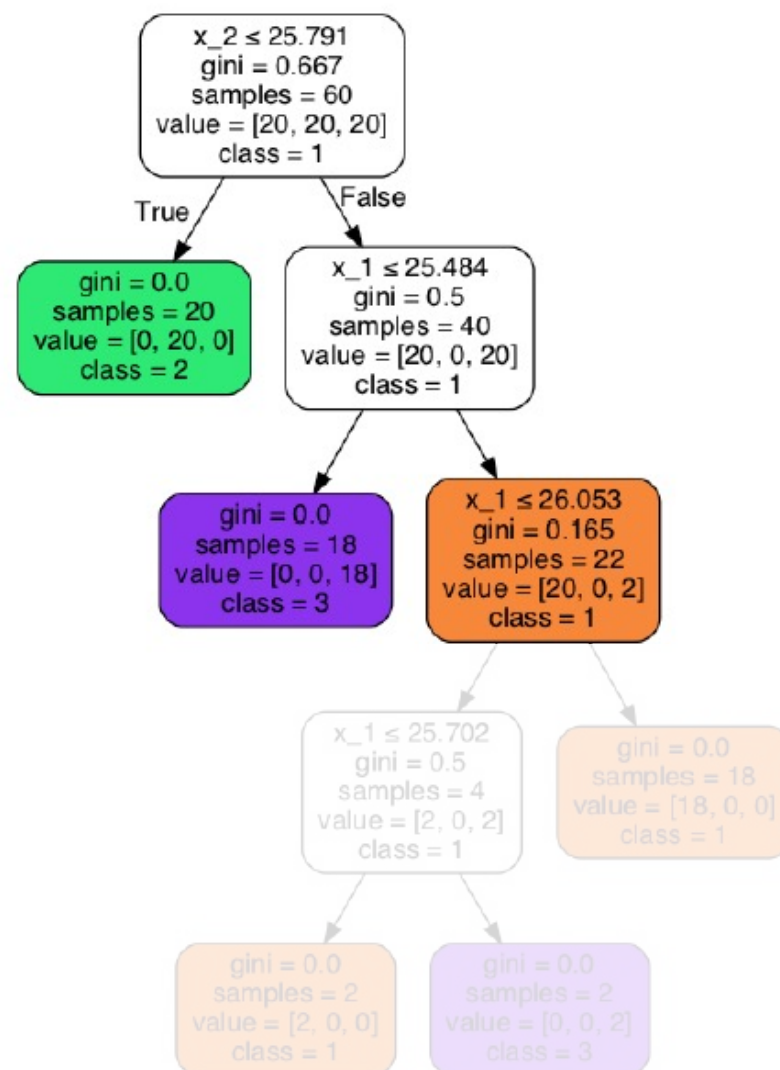
Decision Trees – Address Overfitting

Reduced Error Pruning

Growing a decision tree fully, then removing branches without reducing predictive accuracy, measured using a *validation set*.

- ▶ Start at leaf nodes
- ▶ Look up branches at last decision split
- ▶ replace with a leaf node predicting the majority class
- ▶ If validation set classification accuracy is not affected, then keep the change

This is a simple and fast algorithm that can simplify over complex decision trees



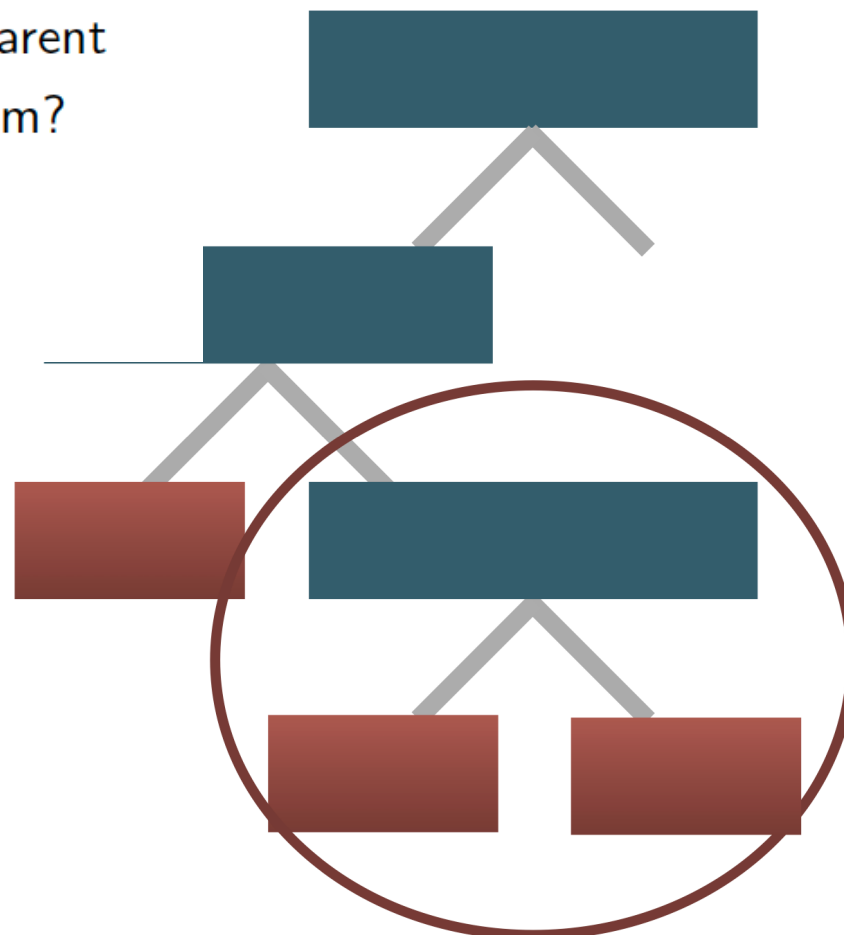
Decision Trees – Address Overfitting

Entropy Based Pruning

Grow a decision tree fully, then

- ▶ Chose a pair of leaf nodes with the same parent
- ▶ What is the entropy gain from merging them?
- ▶ If lower than a threshold, merge nodes

This doesn't require additional data



Decision Trees – Ensemble methods

Bagging: Bootstrap aggregating

Uniformly sample initial dataset with replacement into m subsets

For example:

if data set has 5 samples, $(s_1, s_2, s_3, s_4, s_5)$

make a whole bunch of similar data sets:

▶ $(s_5, s_2, s_2, s_1, s_5)$

▶ $(s_4, s_2, s_1, s_3, s_3)$

▶ $(s_2, s_5, s_3, s_1, s_1)$

▶ $(s_3, s_1, s_2, s_4, s_4)$

Train a different decision tree on each set

To classify, apply each classifier and choose the correct one by majority vote

Decision Trees – Ensemble methods

Boosting - Kearns and Valiant (1988):

“Can a set of weak learners create a single strong learner?”

We make a weighted sum of very weak learners

- so long as they all learn different things then it works!

AdaBoost:

- ▶ Train a weak learner on one sampled data set
- ▶ See what it does well on
- ▶ Weight the difficult data samples more
- ▶ repeat

This makes a series of weak learners that have learned how to use given features to classify easy and hard samples more correctly

Decision Trees – Ensemble methods

Gradient boosting with trees - Friedman (1999):

Generalise Adaboost to Gradient boosting to handle any loss function

Shortcomings are where the residuals are larger

So fit a tree to the residuals:

▶ $x_1, y_q - f(x_1)$

▶ $x_2, y_q - f(x_1)$

▶ $x_3, y_q - f(x_1)$

▶ $\vdots$

▶ $x_n, y_q - f(x_n)$

more detail available at

http://www.chengli.io/tutorials/gradient_boosting.pdf

Decision Trees – Ensemble methods

Random Forests

Apply bagging

but when learning the tree for each subset, chose the split by searching over a random sample of the features

Reduces overfitting

Decision Trees – Summary

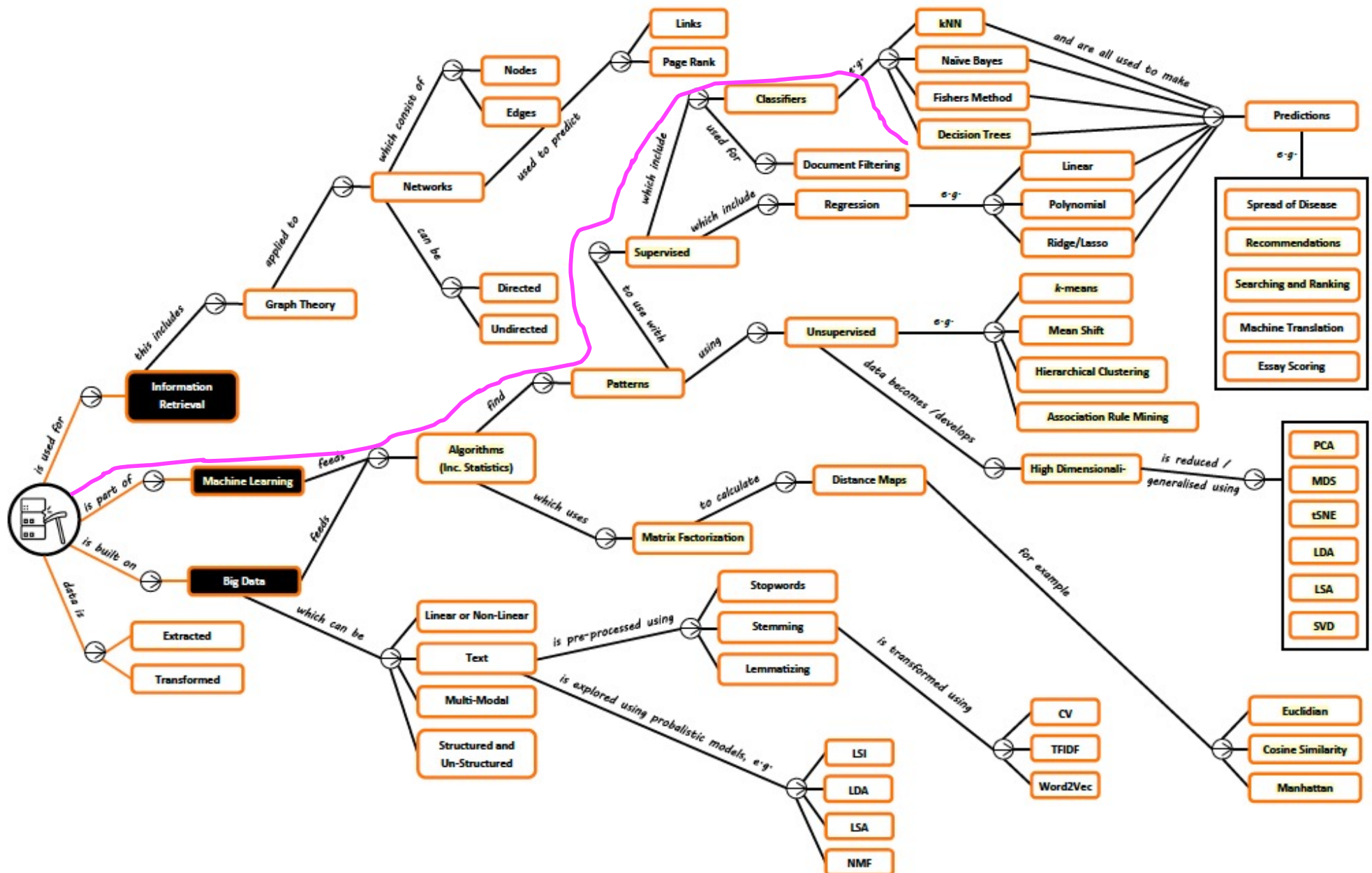
Advantages:

- ▶ Interpretability
- ▶ Ability to work with numerical and categorical features
- ▶ Good with mixed tabular data

Disadvantages:

- ▶ Might not scale effectively for lots of classes
- ▶ Features that interact are problematic

Decision Trees – Roadmap



Decision Trees – Textbook

PART FOUR CLASSIFICATION

The classification task is to predict the label or class for a given unlabeled point. Formally, a classifier is a model or function M that predicts the class label $\hat{y}$ for a given input example $\mathbf{x}$, that is, $\hat{y} = M(\mathbf{x})$, where $\hat{y} \in \{c_1, c_2, \dots, c_k\}$ and each c_i is a class label (a categorical attribute value). Classification is a *supervised learning* approach,

- Data Mining and Machine Learning: Fundamental Concepts and Algorithms M. L. Zaki and W. Meira

4 Classification: Alternative Techniques

The previous chapter introduced the classification problem and presented a technique known as the decision tree classifier. Issues such as model overfitting and model evaluation were also discussed. This chapter presents alternative techniques for building classification models—from simple techniques such as rule-based and nearest neighbor classifiers to more sophisticated techniques such as artificial neural networks, deep learning, support vector machines, and ensemble methods. Other practical issues such as the class imbalance and multiclass problems are also discussed at the end of the chapter.

- Introduction to Data Mining, *P. Tan et al*

Decision Trees – Learning Outcomes

- **LO1:** Demonstrate an understanding of decision tree fundamentals, such as (exam)
 - Calculating impurity and constructing a decision tree using algorithms like ID3, C4.5, etc, given a dataset and distance metric
 - Addressing overfitting in decision trees like pruning
 - Understanding ensemble methods using decision trees like bagging, boosting, random forests
- **LO2: Implement** the learned algorithms using decision trees (course work)

Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: <https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf>*
- *ChatGPT or other AI-based techs*